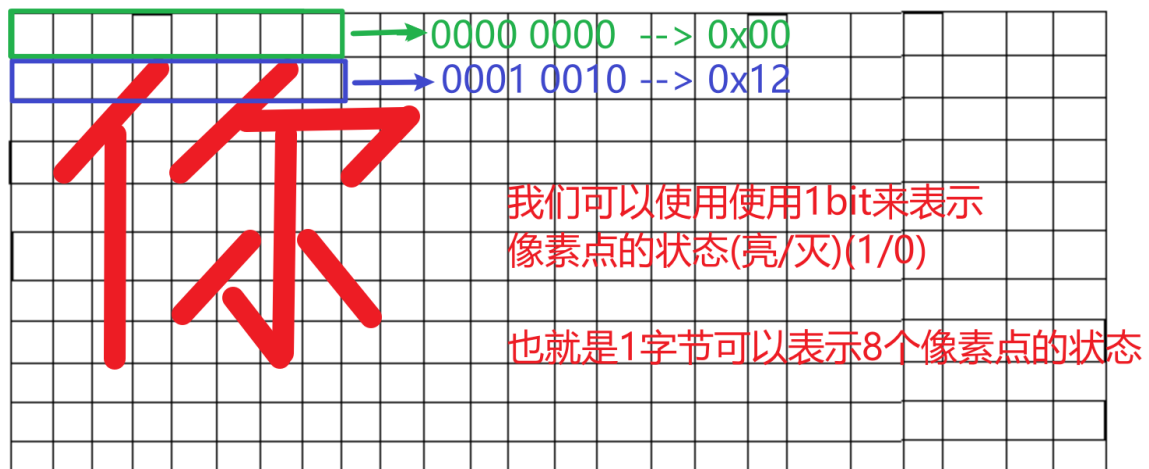


一、文字显示

1.1 字符取模软件的使用



在屏幕上显示字符，本质就是按照一定的规则显示对应的像素点

点阵液晶取模

该软件 把字符按照一定的规律 变成16进制的数据

(1) 参数设置

字体、字号

其他选项 ----> 取模方式: 横向取模



(2) 在文字输入区

输入你想要显示的字符 ---》输入完成之后按 Ctrl + Enter 进行确认

(3) 取模方式

选择C51

(4) 点阵生成区，生成了对应的数据

```

1  /*-- 文字： 徐  --*/
2  /*-- 微软雅黑14； 此字体下对应的点阵为： 宽x高=19x25  --*/
3  /*-- 宽度不是8的倍数，现调整为： 宽度x高度=24x25  --*/
4  0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x0C,0x1C,0x00,0
  x1E,0x3C,0x00,0x1C,0x3E,0x00,0x38,0x77,0x00,0x70,0xE3,0x00,0x6F,0xE3,0x80,0x
  EF,0xC1,0xE0,0x1F,0xFF,0xE0,0x38,0x1C,0x00,0x38,0x1C,0x00,0x78,0x1C,0x00,0xF
  B,0xFF,0xE0,0xF8,0x1C,0x00,0x39,0xDF,0x00,0x3B,0x9F,0xC0,0x3F,0x9D,0xE0,0x3F
  ,0xDC,0xE0,0x38,0xF8,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
5
6
7  一个数据就是一个字节，一个字节占8bit，可以描述8个像素点的亮灭状态

```

1.2 显示

是取模的逆操作，把数据还原成字符

(1) 首先把数据保存起来，数组保存

```

1  char word[宽度*高度/8] = {xxx};

```

(2) 显示

```

1  解析每一个bit，如果该bit是1， 显示对应的颜色
2  使用 &1 来判断该bit是否为0
3
4      xxxx xxxY
5  &  0000 0001
6  -----
7      0000 000Y
8  如果按位运算之后的结果为真(非0)，则表示对应的像素点需要进行显示
9  一个一个bit进行判断

```

二、触摸屏

Linux触摸屏事件，在系统中所有输入事件存储在输入子系统中，输入设备(键盘、鼠标、触摸屏)都对应一个文件名。

触摸屏的文件名 /dev/input/event0

所有的输入事件保存在一个结构体中

input_event

保存 /usr/include/linux/input.h

```

1  struct input_event {
2      struct timeval time; //该事件发生的时间
3
4      __u16 type; //事件类型
5          EV_KEY  按键事件
6          EV_REL  鼠标事件
7          EV_ABS  触摸屏事件
8
9      __u16 code; //code根据事件的不同有不同的含义
10         if(type == EV_KEY)  code为按键的键值
11             #define BIN_TOUCH 0x14a //触摸屏按键
12         if(type == EV_ABS)  code是标识符

```

```

13         ABS_X --> 表示当前value的值为触摸点横坐标的数据
14         ABS_Y --> 表示当前value的值为触摸点纵坐标的数据
15
16         __s32 value; //根据事件类型的不同有不同的含义
17         if(type == EV_KEY) value 有两个值 1/0 按键按下/按键抬起
18         if(type == EV_ABS) value根据code的值来表示具体的坐标数据
19     };
20
21 获取触摸点坐标:
22     type == EV_ABS && code == ABS_X ==> x = value
23     type == EV_ABS && code == ABS_Y ==> y = value
24
25 触摸屏的坐标范围:
26     x [0, 1024)
27     y [0, 600)
28
29 和显示屏不一样
30 对于板子的同一个位置
31     x像/800 = x触/1024
32     y像/480 = y触/600
33
34 也可以如下近似转换:
35     x像 = x触/1.25
36     y像 = y触/1.25

```

作业:

手动控制图片切换

大家参考左右滑动, 完善程序, 实现上下滑动的判断

左右滑动控制图片切换(上一张、下一张)